Benchmarking Spiking Reservoir Architectures for Efficient Univariate Time Series Classification

Favour Akpasi

***Abstract*—Spiking Neural Networks (SNNs) offer a promising avenue for energy-efficient processing on edge devices; however, they are often difficult to train. Reservoir Computing (RC) mitigates training complexity by fixing internal weights, restricting training to a read-out layer. The combination of RC and SNN promises energy efficient information processing that mitigates training complexity. This study presents a benchmark of reservoir-based spiking architectures against a standard Long Short-Term Memory (LSTM) network for the classification of electrocardiogram signals. We evaluated three distinct architectures: a baseline LSTM, a stochastic Liquid State Machine (LSM) with fixed random connectivity, and a structured Legendre Spiking Neural Network (LSNN) utilizing deterministic weights derived from the Legendre Memory Unit (LMU) theory. Through a grid search of critical hyperparameters—specifically the input scale and decay rate (β) for the LSM, and the Legendre dimension (d) for the LSNN—we analyze the trade-offs between classification accuracy, computational latency, spike sparsity and the impact of the reservoir's initialization strategy. Experimental results demonstrate that a structured LSNN ($d = 24$) achieves a peak test accuracy of 97.07%, closely matching the LSTM baseline of 97.24% and outperforming the random LSM at 89.96%. Analysis of spiking dynamics reveals that the LSNN leverages a dense-sparse coding strategy (71.26% sparsity) to encode temporal dependencies, while the LSM's higher sparsity (89.25%) correlates with insufficient information retention. While simulation on Von Neumann hardware resulted in higher latency for spiking models (30–40 ms/sample) compared to the LSTM (6.2 ms/sample), the high sparsity confirms the theoretical potential for significant energy savings on event-driven neuromorphic hardware. These findings indicate that full backpropagation through time is not strictly necessary for this task and incorporating mathematical structure into spiking reservoirs bridges the performance gap with conventional deep learning, enabling high-accuracy temporal processing compatible with the strict constraints of edge computing.**

***Index Terms*—Reservoir Computing, Spiking Neural Networks, ECG Classification, Liquid state machine (LSM), LSNN, Legendre Polynomials, Edge AI, LSTM.**

I. INTRODUCTION

THE processing of temporal data, such as electrocardiogram (ECG) signals, presents challenges in maintaining memory over time while minimizing computational costs. Although conventional Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks, have excelled for such tasks because of their ability to mitigate vanishing gradients, they are computationally intensive. This high resource demand hinders their deployment on edge devices with strict size, weight, and power constraints [1]–[3].

To address this, we investigate Spiking reservoir architectures. RC systems map inputs into a high-dimensional space through a fixed, non-linear "reservoir," training only a linear readout layer [2]. This study benchmarks a conventional LSTM against two spiking reservoir models: the stochastic Liquid State Machine (LSM) and the structured Legendre Spiking Neural Network (LSNN). We used the ECG5000 dataset to evaluate whether structured spiking reservoirs can match the high accuracy of LSTMs while offering the architectural efficiency inherent to neuromorphic approaches.

II. METHODOLOGY

A. Dataset

We utilize the ECG5000 dataset from the UCR Time Series Archive. It consists of 5000 electrocardiogram sequences, each sampled at 140 Hz over a 1-second duration (140 time steps). The raw dataset contains five distinct classes that represent different types of heartbeat. The initial exploratory analysis revealed an extreme class imbalance. Class 1 ("Normal") constitutes approximately 58.4% of the data, while classes 3, 4, and 5 are severely underrepresented (see Table I). To ensure a proper evaluation of the capabilities of the architectures' capabilities rather than their handling of imbalances, and following precedents in the literature [2], the problem was transformed into a binary classification task. For SNN models, data was passed sequentially over time steps to mimic the temporal processing of biological neurons.

- **Normal:** Original Class 1 retained as Class 0.
- **Abnormal:** Original Classes 2, 3, 4, and 5 aggregated into Class 1.

This results in a more balanced split suitable for benchmarking (see Table II).

The implementation of all models and preprocessing steps is publicly available in a GitHub repository. [1]

B. Model Architectures

1) Model A: Long Short-Term Memory (LSTM): A standard LSTM network was established using PyTorch to serve as the accuracy baseline. The LSTM utilizes gating mechanisms to control information flow, allowing it to learn long-term dependencies [1]. Each input sample is represented as $\boldsymbol{x} \in \mathbb{R}^{T\times 1}$. The model consists of a 2-layer LSTM with hidden state dimensionality $h = 64$ (using `batch_first`

Manuscript submitted for a practical work as part of a Master's degree program. This work explores structured and unstructured Reservoir based spiking models for energy-efficient univariate time series classification tasks.

Favour Akpasi is a student of Johannes Kepler Universität Linz, Austria. (e-mail: akpasifavour@gmail.com)

[1] https://github.com/FavourAkpasi/Benchmarking-Spiking-Reservoir-Architectures

TABLE I
ORIGINAL DATASET CLASS DISTRIBUTION

Class	Training set	Test set
N	58.4% (292)	58.38% (2627)
R-on-T PVC	35.4% (177)	35.33% (1590)
PVC	2.00% (10)	1.91% (86)
SP	3.80% (19)	3.89% (175)
Unclassified beat (UB)	0.40% (2)	0.49% (22)

Note: The class distribution of the ECG5000 Dataset before binary processing. Abbreviations — N: Normal; R-on-T PVC: R-on-T premature ventricular contraction; PVC: Premature ventricular contraction; SP: Supra-ventricular premature beat; UB: Unclassified beat.

TABLE II
BINARY CLASS DISTRIBUTION AFTER CLASS AGGREGATION

Class	Training set	Test set
Normal (Class 0)	58.4% (292)	58.38% (2627)
Abnormal (Class 1)	41.6% (208)	41.62% (1873)

Note: The class distribution of the ECG5000 Dataset after binary processing. Class 0 corresponds to Normal beats (original Class 1), while Class 1 aggregates R-on-T PVC, PVC, SP, and UB (original Classes 2–5).

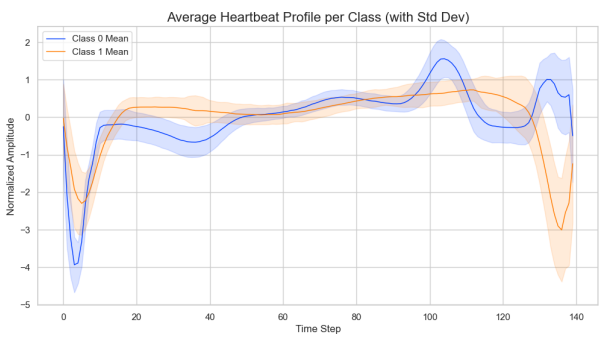

Fig. 1. Average heartbeat profile per class (mean ± 1 standard deviation) from the processed dataset. The plot shows the class-wise mean normalized sample over the 140 time steps, with the shaded regions indicating one standard deviation across samples. The shape differences between the classes visually validate the feasibility of binary classification.

formatting). At each forward pass, the initial hidden and cell states are initialized to zeros. The LSTM produces a sequence of hidden outputs. We retain the final output as a fixed-length representation of the entire sequence. This representation is passed to a linear readout layer to produce class logits. The network is trained using a cross-entropy objective on the logits and tuned to achieve a target accuracy of approximately 96%, serving as a benchmark for the spiking models.

2) Model B: Liquid State Machine (LSM): The LSM is a reservoir computing architecture that processes data using a recurrent network of spiking neurons followed by a trainable linear readout layer [4]. The reservoir is composed of N Leaky Integrate-and-Fire (LIF) neurons. The connections are defined by the input weight matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d\times N_{in}}$ and the recurrent weight matrix $\mathbf{W}_{\text{rec}} \in \mathbb{R}^{d\times d}$. The weights are initialized randomly and remain frozen during training to preserve the separation property and the fading memory of the system [4], [5].

The dynamics of the reservoir are governed by the discrete-time LIF equations. At each time step t, the total synaptic current $\boldsymbol{u}_t$ is computed by integrating external inputs and recurrent spikes from the previous step:

$$\mathbf{u}_t = \mathbf{x}_t W_{\text{in}}^\top + \mathbf{s}_{t-1} W_{\text{rec}}^\top \tag{1}$$

where $\boldsymbol{x}_t \in \mathbb{R}^{B\times N_{in}}$ is the input batch and $\mathbf{s}_{t-1} \in \{0,1\}^{B\times d}$ is the spike vector from the previous step.

The membrane potential $\mathbf{m}_t$ evolves according to a decay dynamics governed by the factor β:

$$\boldsymbol{m}_t = \beta \boldsymbol{m}_{t-1} + (1-\beta)\boldsymbol{u}_t - \boldsymbol{s}_{t-1} V_{th} \tag{2}$$

Here, $\beta \equiv e^{-\Delta t/\tau_m}$ represents the decay rate associated with the membrane time constant τ_m [6]. When the membrane potential exceeds the firing threshold V_{th}, a spike is emitted ($\mathbf{s}_t = 1$), and the potential is reset. To translate the temporal spike trains into a static representation for classification, we utilize a rate-coding readout scheme. The spike trains are aggregated over the time window T to form a mean firing rate vector $\mathbf{r} \in \mathbb{R}^{B\times d}$. Finally, the class logits $\mathbf{z} \in \mathbb{R}^{B\times C}$ are computed using a trainable linear readout:

$$\mathbf{z} = \boldsymbol{r}\mathbf{W}_{\text{out}}^\top + \boldsymbol{b}_{\text{out}} \tag{3}$$

where $\mathbf{W}_{\text{out}}$ and $\mathbf{b}_{\text{out}}$ are the only trainable parameters in the network and C is the number of classes.

3) Model C: Legendre Spiking Neural Network (LSNN): The LSNN is a hybrid architecture that combines a non-spiking Legendre Delay Network (LDN) for temporal feature extraction with a spiking neural network for classification. The LDN layer approximates the input history using Legendre polynomials, efficiently compressing temporal information into a state vector of dimension d [2].

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1}\bar{\mathbf{A}}^\top + \boldsymbol{x}_t\bar{\mathbf{B}}^\top \in \mathbb{R}^{B\times d} \tag{4}$$

where $\mathbf{h}_t$ is the zero initialized memory state. $\bar{\mathbf{A}} \in \mathbb{R}^{d\times d}$ and $\bar{\mathbf{B}} \in \mathbb{R}^{d\times 1}$ are the discretized versions of the continuous Legendre state-space matrices, calculated with a fixed time-step Δt. These weights are fixed (frozen) during training to ensure optimal orthogonal projection of the sliding window history. The continuous-valued features $\boldsymbol{h}_t$ are projected into a hidden layer of spiking neurons. The feature dimension is expanded to a hidden dimension of $3d$ [2]. A trainable encoder matrix $W_{\text{enc}} \in \mathbb{R}^{(3d)\times d}$ maps the Legendre features to synaptic currents:

$$\boldsymbol{u}_t = \boldsymbol{h}_t\mathbf{W}_{\text{enc}}^\top \in \mathbb{R}^{B\times 3d} \tag{5}$$

The hidden layer consists of Leaky Integrate-and-Fire (LIF) neurons. The membrane potential $\mathbf{m}_t$ and the spike output $\boldsymbol{s}_t$ is then:

$$(\boldsymbol{s}_t, \boldsymbol{m}_t) = \text{LIF}_\beta(\boldsymbol{u}_t, \boldsymbol{m}_{t-1}) \tag{6}$$

where β is the decay rate and $\boldsymbol{s}_t \in \{0,1\}^{B\times 3d}$ is the spike vector. To perform classification, we aggregate the spike trains into a rate-coded representation $\boldsymbol{r} \in \mathbb{R}^{B\times 3d}$ by averaging over the time dimension T. Finally, the class logits $\boldsymbol{z} \in \mathbb{R}^{B\times C}$ are obtained using a trainable linear readout layer:

$$\boldsymbol{z} = \boldsymbol{r}\mathbf{W}_{\text{out}}^\top + \boldsymbol{b}_{\text{out}} \tag{7}$$

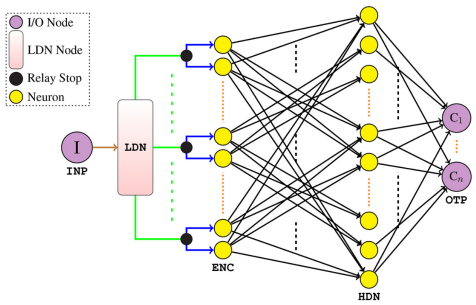

Fig. 2. LSNN model architecture (all neurons are IF spiking). The INP is connected to the LDN through a static connection weight of 1. The outputs from the LDN, i.e., x(t) (the green connections) are relayed as is to the relay stops where each individual feature signal is multiplied by the static weights of 1 and 1 (simultaneously), i.e., the blue connections and fed to ENC. Only the connections in black are trained. adapted from [2]

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{C\times 3d}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{C}$ are the trainable parameters.

III. Experimental Setup

A. Hardware Environment

All experiments were conducted on an Apple MacBook Pro (M2, 16GB Unified Memory) which uses the ARM64 architecture and Apple's Metal Performance Shaders acceleration. This machine behaves differently from the NVIDIA GPUs (CUDA) used in most standard literature. The models were implemented in Python 3.9 using PyTorch 2.0. All models were trained using the Adam optimizer with CrossEntropyLoss for 30 epochs and we performed a grid search to optimize the important parameters for the spiking architectures.

B. Hyperparameter Optimization

We perform a grid search to optimize the parameters that govern the dynamics of the spiking reservoirs. The Learning Rate (LR) was varied between [0.001, 0.005] to ensure convergence. For the LSM, which relies on the so called "edge of chaos" transition to separate inputs, we tuned the Input Scale (controlling the magnitude of current injection) and the membrane decay rate β. The Input Scale was tuned in [10.0, 50.0] to balance reservoir silence against saturation, while β was tuned in [0.9,0.95] to optimize the fading memory duration [3], [4]. For the LSNN, the critical structural parameter is the Legendre Dimension (d), which determines the dimensionality of the subspace onto which the history of the input is projected. We tested [6, 12, 24] to evaluate how memory capacity scales with reservoir complexity. The simulation time window was fixed to 140 steps to match the temporal resolution of the ECG signals [2], [3], [7]. The comparative performance of the top configurations is detailed in Table III.

C. Sparsity and Event Metrics

An advantage of SNNs is their ability to process information sparsely, communicating only when significant events occur. To quantify this, we accumulated the output spike tensors for every sample in the test set. We defined Spike Density as the fraction of the total possible spike events $(neurons \times timesteps)$ that actually caused a spike. Sparsity was calculated as $(1 - SpikeDensity) \times 100$. Additionally, we tracked Spikes Per Sample to estimate the total synaptic workload required for a single inference. These metrics serve as a proxy for dynamic power consumption on neuromorphic hardware, where energy is only consumed during spike events [6], [8].

TABLE III
Grid Search Summary

Model	Hyperparameters	LR	Accuracy (%)
LSNN (Structured)	$d = 24$, Gain=1.0	0.005	**97.07**
LSNN (Structured)	$d = 12$, Gain=1.0	0.005	93.84
LSNN (Structured)	$d = 6$, Gain=1.0	0.005	89.76
LSM (Random)	Scale=10.0, $\beta = 0.9$	0.005	89.96
LSM (Random)	Scale=50.0, $\beta = 0.9$	0.005	87.49

Note: The summary of the grid search results. LR denotes Learning Rate. For the Structured LSNN, d represents the dimensionality of the Legendre memory basis. For the Random LSM, Scale refers to the input weight scaling factor, and β indicates the membrane potential decay rate of the LIF neurons.

D. Latency and Computational Cost

To observe and benchmark computational overhead, we measured both training time and inference latency. Training time was recorded for the entire 30-epoch cycle. For inference latency, we measured the wall-clock time required to process a single sample. To ensure robust measurements, we utilized a "warmup" phase to stabilize the GPU state, followed by repeated forward passes to calculate the average millisecond (ms) latency per sample. This approach highlights the distinction between algorithmic time steps and hardware execution time [7], [9]

IV. Results and Discussion

A. Classification Accuracy

From the results on Table IV, we notice the Structured LSNN achieved a maximum accuracy of 97.07% (with d=24), closely matching the Baseline LSTM (97.24%). In contrast, the Random LSM achieved 89.96%. This disparity highlights the efficacy of the LDN structure in the LSNN; by mathematically deriving weights to orthogonalize the input history, the LSNN captures long-term dependencies that the random projections of the LSM fail to retain. The LSM's reliance on random connectivity necessitates a much larger reservoir to achieve the separation property required for high accuracy, whereas the LSNN achieves superior results with a compact and structured formulation [2].

B. Sparsity

The spike metrics as seen in Table IV reveal a significant trade-off between sparsity and accuracy.

1) LSM: : Exhibited high sparsity (89.25%) and low spike density (0.107). Although theoretically energy-efficient, this extreme sparsity implies that the reservoir often failed to propagate sufficient information through time, leading to lower accuracy.

TABLE IV
COMPREHENSIVE PERFORMANCE BENCHMARK: NON-SPIKING BASELINE VS. SPIKING RESERVOIRS

Model Architecture	**Accuracy** (%)	**Training Time** (s)	**Inference Latency** (ms)	**Spikes / Sample** (Count)	**Sparsity** (%)
Model A: LSTM (Baseline)	97.24	18.74	6.20	N/A	N/A
Model B: LSM (Random)	89.96	78.82	30.64	963	89.3
Model C: LSNN (Structured)	97.07	100.54	40.01	2,897	71.3

Note: The table shows comprehensive benchmark comparing spiking models across performance, and efficiency metrics. Training time is measured in seconds and Inference latency is measured in miliseconds.

2) LSNN: : Displayed moderate sparsity (71.26%) and higher spike density (0.287). The LSNN generated approximately 3x more spikes per sample (2897 vs. 962) than the LSM. This increased activity in the LSNN suggests that the structured weights successfully drove the neurons to encode the continuous-time history of the ECG signal into dense spike trains [7]. While this results in higher theoretical energy consumption than the LSM, it is still significantly more efficient than the continuous-valued activations of the LSTM, which effectively has 0% sparsity (processing dense floating-point numbers at every step).

C. Latency and the Simulation Bottleneck

The latency results as seen in Table IV illustrate the so called "timescale mismatch" challenge when simulating SNNs on Von Neumann architectures [10]. The SNN models were approximately 5 to 6 times slower than the LSTM during inference. This is consistent with other literature noting that SNNs are computationally expensive to simulate on standard GPUs because the differential equations for every neuron must be solved iteratively for every time step (T=140), while LSTMs are highly optimized for parallel matrix operations [4]. The LSNN's higher latency compared to the LSM is attributed to the additional matrix multiplications required for the Legendre projection at each step [2]. However, it is important to note that on dedicated neuromorphic hardware (e.g., Loihi, SpiNNaker), this relationship often inverts, as the hardware processes spikes asynchronously and in parallel, decoupling execution time from the number of simulation steps [9], [11].

V. CONCLUSION

This study compared the effectiveness of reservoir-based Spiking Neural Networks with standard Deep Learning for time-series classification. We demonstrated that the Legendre Spiking Neural Network (LSNN) successfully bridges the performance gap, achieving 97.07% accuracy, which is relatively close the LSTM baseline compared to the the 89.96% achieved by the random Liquid State Machine (LSM). Our results confirm that integrating mathematical structure (Legendre polynomials) into the reservoir allows for robust temporal feature extraction. Although SNNs incur higher latency and training times on standard GPU hardware due to simulation overhead, the sparsity analysis (71-89% silence) indicates massive potential for energy savings if deployed on event-driven neuromorphic processors. The LSNN effectively leverages Spikes, Sparsity, and Static suppression to encode complex temporal dependencies. Future work will focus on implementing the LSNN on neuromorphic hardware platforms such Intel Loihi to validate whether the theoretical efficiency gains translate to physical energy reductions.

REFERENCES

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] R. Gaurav, T. C. Stewart, and Y. Yi, "Reservoir based spiking models for univariate time series classification," *Frontiers in Computational Neuroscience*, vol. 17, 2023. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fncom.2023.1148284

[3] A. Fountain and C. Merkel, "Effect of biologically-motivated energy constraints on liquid state machine dynamics and classification performance," *Neuromorphic Computing and Engineering*, vol. 2, no. 2, p. 024005, apr 2022. [Online]. Available: https://doi.org/10.1088/2634-4386/ac5d1f

[4] V. Kholkin, O. Druzhina, V. Vatnik, M. Kulagin, T. I. Karimov, and D. N. Butusov, "Comparing reservoir artificial and spiking neural networks in machine fault detection tasks," *Big Data Cogn. Comput.*, vol. 7, p. 110, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:259241463

[5] H. Jaeger, "The "echo state" approach to analysing and training recurrent neural networks-with an erratum note," *GMD Report*, vol. 148, 2001.

[6] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[7] A. Voelker, I. Kajić, and C. Eliasmith, "Legendre memory units: Continuous-time representation in recurrent neural networks," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, Eds., 2019, pp. 15 570–15 579.

[8] S. Dey, D. Banerjee, A. M. George, A. Mukherjee, and A. Pal, "Efficient time series classification using spiking reservoir," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

[9] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, M. Lolis, A. Lu, A. McCoy, R. Paul, J. Tse, G. Vigh, B. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[10] T. Liu, "Harnessing slow dynamics in neuromorphic computation," 2019. [Online]. Available: https://arxiv.org/abs/1905.12116

[11] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The spinnaker project," *IEEE Transactions on Computers*, vol. 62, no. 11, pp. 2454–2467, 2013.